Evolution 1 · Lecture

The Agent Loop

From a model call to an agent: chain-of-thought, ReAct, the loop without a halt, and the loop with a ledger

SURVEY | BUILD | GOVERN | DEFEND

CIDM 6333 · Agentic Systems Design and Development
Survey: ReAct · AutoGPT/BabyAGI · gzkit genesis · Build: a governed loop that halts; PRD attested

Sixty minutes, five moves

0:00–0:02

The run of show

1	0:00	**Framing**	A loop without a halt is a bill; what E0 gave us; what a loop must have
2	0:10	**Spine**	From prompting to programs: chain-of-thought → ReAct; the minimal loop; structured outputs; halt conditions and budgets
3	0:30	**Teardowns**	ReAct and AutoGPT/BabyAGI — a live Design Analysis on the board
4	0:45	**Reference design**	gzkit's genesis: the loop plus the thing the loop lacked; the PRD as WHAT; the first ledger events
5	0:52	**Evaluation lens**	What a strong Sortie 1 looks like; Flight Card 1 brief (center of envelope)

A loop without a halt is a bill

0:02–0:10

The one design lesson of 2023, stated up front

Autonomy without a stop condition is not a feature. It is an open account with the model provider.

What E0 gave us

The model has no persistent state, no ground truth, no accountability. The loop inherits all three gaps unless designed out: a budget (state), an observation (ground truth), a ledger (accountability).

Tonight's claim, which Sortie 1 will try to disprove
The loop halts on every input class, within budget, every turn in the ledger.
False if a run exceeds the turn budget, halts with no ledger event, or answers without a reasoning turn.

From prompting to programs (2020–2022)

0:10–0:15

How a prompt became a control structure

GPT-3 few-shot (2020)
Examples in the prompt steer the completion. The prompt is data.

Chain-of-thought (Wei et al., 2022)
'Let's think step by step.' Reasoning becomes an artifact you can read.

Self-consistency (2022)
Sample several chains, vote. The first evaluator, and the first cost multiplier.

ReAct (Yao et al., 2022)
Interleave reasoning with actions and observations. The prompt is now a loop body.

Why this matters for the build

Chain-of-thought is planning without commitment. Nothing checks the reasoning against the world.
ReAct adds the observation. The next reasoning step has ground truth in it.
An observation inside the loop makes a generative system an agent.

ReAct, read as a design document

0:15–0:20

Thought → Action → Observation, until a final answer

Thought 1	I need to find when the bridge opened, then compute the years since.
Action 1	`search["Golden Gate Bridge opened"]`
Observation 1	The bridge opened to traffic on May 28, 1937.
Thought 2	Opened 1937; the current year is 2026; 89 years.
Action 2	`finish[89 years]`

Illustrative trace in ReAct's format; the paper's tasks were HotpotQA, FEVER, ALFWorld, WebShop.

Component map

Foundation PaLM-540B / GPT-3, hosted
Perception question + observations
Planning free-text thoughts between actions
Tools fixed set (search, lookup, finish)
Memory the trace only
Coordination single agent
Evaluation none in-loop
Governance prompt author, then absent
Runtime script; halt = finish

Lacked: memory, a budget, verification.

The minimal loop you will build this week

0:20–0:25

Messages, a model call, a tool-or-answer decision, an observation, repeat — with a halt

```
messages = [system, user]
for turn in range(BUDGET):                  # halt: turn budget
    step = model(messages, schema=Step)     # structured output
    ledger.append("turn", step)             # ledger first
    if step.kind == "answer":               # halt: final answer
        return step.text
    if step.kind == "tool":
        obs = tools.call(step.name, step.args)   # observation
        messages.append(observation(obs))
    if cost.spent > COST_LIMIT:             # halt: cost budget
        return halt("budget")
raise Halt("turn budget exhausted")         # halt: never silent
```

Structured output

Step is a Pydantic model: kind ∈ {answer, tool}, name, args, text. A malformed decision is rejected; that is an observation too.

Four halts

Final answer · turn budget · cost budget · error. Each lands in the ledger with a reason. A silent exit is the failure.

Two foundations

Same loop against a hosted API and Ollama's endpoint. Measure latency, cost, and schema compliance.

Halt conditions, budgets, and day-one failures

0:25–0:30

Each failure becomes a falsifier on the card

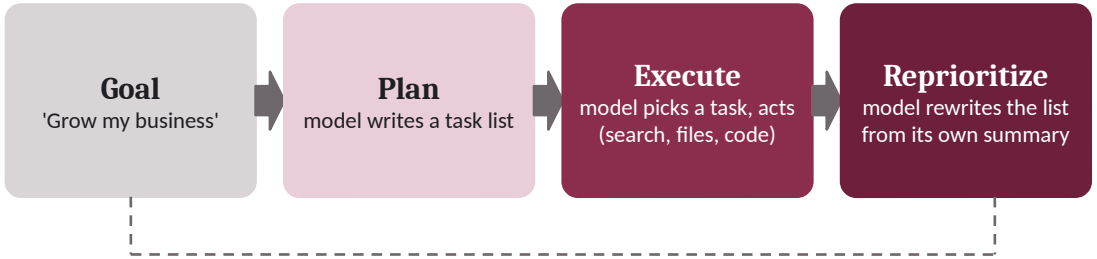
Failure	What you will see	Design response
Loop that does not halt	Model never emits 'answer'; keeps calling the same tool	Turn budget; detect repeated identical actions; halt with reason
Silent exception	Tool raises; loop swallows it; run 'succeeds' with nothing done	Errors are observations; error count is a halt; ledger records it
Format the small model cannot produce	Local 8B model emits prose instead of the Step schema	Parser rejects; retry budget; then halt('format'); log as squawk, not FAIL
Answer without reasoning	Model jumps to 'answer' on turn 1 for a question that needed a tool	Falsifier: a final answer with no reasoning turn preceding it
Adversarial input	Prompt asks the loop to ignore its budget or system prompt	Budget is enforced in code, not in the prompt; the loop halts anyway

Rule for the semester: a budget the model can talk itself out of is not a budget. Halts live in code and the ledger.

Teardown: AutoGPT and BabyAGI (spring 2023)

0:30–0:38

The loop with goals and no grounding, budget, or evaluator



...and around again, indefinitely: the only halt is the human closing the terminal (or the API key running dry)

Which absence caused the spiral?
Evaluation: nothing checked a result before the next plan.
Governance: one-sentence goal; no performance measure (PEAS's P), no halt.
Memory: a vector store of its own outputs; hallucinations retrieved as truth.
Planning was present. Fluency without grounding is a spiral.

Why it mattered anyway

It named the ambition: a goal, then let it run.
It produced the first shared failure record: budget burned, plans rewritten, progress invented.
Every 2024–25 design replies to this spring.
Judgment loop: decision = full autonomy; cost = grounding; failure = spiral; preservation = none.

Live Design Analysis: ReAct vs. AutoGPT

0:38–0:45

Same loop, one year apart

Component	ReAct (2022)	AutoGPT (2023)
Foundation	PaLM / GPT-3, hosted	GPT-4, hosted; cost unbounded
Perception	Question + observations	Goal + its own summaries + web + files
Planning	Thought per step; no plan object	Explicit task list, rewritten each cycle
Tools	3 fixed (search, lookup, finish)	Open set: web, files, code exec, subagents
Memory	The trace, in-window	Vector store of own outputs
Coordination	Single	Single + spawned 'agents'
Evaluation	None in-loop	None; self-graded from summary
Governance	Prompt author, absent	One-line goal; 'continuous mode'
Runtime	Script; halts on finish	Daemon; halts on Ctrl-C or empty wallet

The why, in one paragraph

ReAct optimized for a benchmark and left everything else out. AutoGPT optimized for ambition and left grounding out. Both lacked a budget, an evaluator, and a record. The next slide's system was started to fill that gap in January 2026.

Reference design: gzkit's genesis (January 2026)

0:45–0:52

The loop, plus the thing the loop lacked — a record of intent that survives the session

The founding exchange (2026-01-12)
The model: for maximizing 'Claudishness,' only specification matters.
The operator: specification, methodology, and governance, all three.

[Quote from genesis.md · verify wording against the source before class.]

What was added to the loop

Not a better model call. A PRD that says WHAT before HOW; an ADR that records the decision and alternatives; a brief with allowed paths; an append-only ledger of every turn, gate, and attestation. Governance arrived as the missing component.

This week's governed path

`gz init --mode lite`
scaffolds the covenant; commit once

`gz prd`
the interview: WHAT the system does, for whom, and what 'done' means — never HOW

`prd_created`
the first ledger event; your loop's first turn lands beside it

`Attest`
Gate 5 at the brief level; you observed the artifacts, then signed

The PRD is WHAT, not HOW

0:52–0:54

The interview you will do before you write the loop

Belongs in the PRD

- Who uses it, and for what
- The question class it answers, and what it refuses
- What 'halted correctly' means per class
- Budgets: turns, cost, latency
- What the ledger must hold
- What the human attests, and when

Does not belong in the PRD

- Which model, SDK, or framework (that is ADR-1, next week)
- The Step schema's field names
- For-loop or state graph
- Any code

A PRD that names a framework pre-decides the ADR without alternatives: post-hoc rationalization.

Evaluation lens: what a strong Sortie 1 looks like

0:54–0:57

The Chase reads the black box, not your story

Weak

- 'It works': one happy-path screenshot
- Budget lives in the system prompt
- Ledger is a print statement
- One model, one route, no measurement
- Falsifiers written after the run

Sound

- Three input classes flown: answerable, unanswerable, adversarial
- Budgets enforced in code; halts carry a reason
- Ledger events for every turn; the Chase can replay
- Hosted and local compared: latency, cost, schema compliance
- Card frozen at Go; evidence slice only to the Chase

Exceptional

- A run designed to fail (budget exhausted), and did, in the ledger
- A malformed local-model output logged as a squawk
- The 'answer without reasoning' falsifier tested with a tempting prompt
- One flight-log paragraph on what the SDK hid vs. raw HTTP

Flight Card 1 brief · governed loop that halts

0:57–1:00

Center of the envelope · fill every field before requesting Go

ENVELOPE	center
DESIGN CLAIM	halts on every input class within budget; every turn in the ledger
SYSTEMS UNDER TEST	loop · Step schema · budget guard · ledger writer · one route
ENTRY CONDITIONS	clean tree · gz init committed · PRD attested · Go recorded
TEST POINTS	1 answerable → turn events, answer event; turns ≤ BUDGET
	2 unanswerable → halt(reason) event; no fabricated answer
	3 adversarial → budget enforced; halt event present
INSTRUMENTATION	1 turn event per call; 1 terminal event (answer|halt); run hash
FALSIFIERS	budget exceeded · halt without ledger event · answer without reasoning turn
ABORT	endpoint unavailable; local model cannot emit schema (squawk, not FAIL)
CHASE	peer, from ledger slice and receipts → PASS / FAIL

Observations are perception. Treat them as untrusted input.

A loop without a halt is a bill. Build one that stops, and can say why.